

THE STRUCTURE OF MATHEMATICAL REALITY

Phase IV — Chapter 12

# Information Theory and the Cost of Description

Lecture

Self-Study Curriculum

You arrived at an idea on your own, years ago, and wrote it down as an intuition without a home: that understanding a system and *compressing* its description are somehow the same act. You noticed that if you know the rule that generated a dataset, you have achieved a compression far more powerful than any statistical encoding, because the rule stands in for the data and for all future data the same process will ever produce. You even asked whether the search space of candidate rules could itself be made variable, which is a live research frontier. That intuition was correct, and this chapter is where it gets a rigorous spine. Information theory is the exact framework in which “understanding equals compression” stops being a metaphor and becomes an equation you can compute.

The previous chapter left you with a debt. Universality said the world is compressible: huge classes of systems collapse onto short descriptions. But it treated compression as something that merely *happens*, and never once measured it. This chapter measures it. It defines information, fixes the hard floor on how far any description can be squeezed, and gives you a currency, the bit, in which the cost of a description, and the cost of a *wrong* description, are both exactly quantifiable.

## What information is

Start from a definition that will feel backwards for one second and then obvious. Shannon defined information as *surprise*. An event that was certain to happen carries no information when it happens; you already knew it. An event you thought was unlikely carries a lot of information when it happens, because it overturns more of your expectation. So the information content of an event has to be a decreasing function of its probability: rarer means more informative. The specific function is

information content of an event of probability  $p = -\log_2(p)$  bits.

The minus sign and the logarithm are not arbitrary. The logarithm is forced by one requirement: information from independent events should *add*. If you learn two independent facts, the total surprise should be the sum of the two surprises. But probabilities of independent events *multiply*. The only function that turns multiplication into addition is the logarithm, so log is not a stylistic choice; it is the unique way to make information additive over independent events. The minus sign is there because probabilities are at most 1, so their logs are at most zero, and we want information to come out positive. Base 2 sets the *unit*: one bit is the information in an event of probability one-half.

This is not a metaphor and not merely a formula. It has an operational meaning:  $-\log_2(p)$  is the ideal number of yes/no questions needed to pin down the event. A fair coin flip carries exactly 1 bit: one question (“heads?”) settles it. A roll of a fair six-sided die carries  $\log_2 6 \approx 2.58$  bits: you cannot always settle it in two questions, and three is usually wasteful, so the honest average cost sits between, at 2.58. The fractional bit is real; it is the average question-count of the best possible questioning strategy, and no strategy does better.

## Entropy: the expected information

A single event’s surprise is a start, but you rarely care about one event. You care about a *source*: a process emitting outcomes with some distribution. The natural summary is the average surprise, weighted by how often each outcome actually occurs. That average is the **entropy** of the distribution:

$$H(X) = -\sum p(x) \log_2 p(x)$$

Read it as the expected value of  $-\log_2 p(x)$ : each outcome contributes its own surprise, weighted by its probability. Entropy is the single most important quantity in this chapter, and its importance

comes from the fact that it answers three different-sounding questions with one number:

- **Uncertainty before observation.** How much do you *not* know about the next outcome? A fair coin ( $H = 1$  bit) leaves you maximally uncertain between two options; a coin that lands heads 99% of the time ( $H \approx 0.08$  bits) leaves you almost certain, so it has almost no entropy.
- **Information gained after observation.** How much do you *learn* when the outcome arrives? Exactly as much as your prior uncertainty, because learning the answer is precisely the removal of that uncertainty.
- **Minimum average description length.** How many bits, on average, must you spend to record the outcome? Also  $H$ .

These are not three facts that happen to share a symbol. They are one quantity seen from three sides, and the equivalence is the conceptual heart of information theory. Uncertainty before, information after, and description cost are the *same thing*, because to describe an outcome is to resolve the uncertainty about it, and resolving uncertainty is gaining information. Hold onto this triple identity; every later idea in the chapter is a consequence of it.

Notice too that entropy is maximized by the *flattest* distribution. Among a fixed number of outcomes, the uniform distribution has the most entropy; any skew toward some outcomes lowers it. A source with structure, one that favors some outcomes, is a source with entropy below the maximum, and that gap below the maximum is exactly the room a compressor has to work in. This is the first quantitative form of the slogan you will leave with: structure is compressibility, and entropy measures how much structure a source lacks.

#### Aside

The whole chapter turns on one inversion. High entropy is not richness; it is the *absence* of exploitable structure, the flat distribution with nothing to lean on. The interesting sources, the ones you can model, are the *low*-entropy ones, whose skew and dependence a short description can capture. When you compress well, you are not adding order; you are cashing in order the source already had.

## Compression and the minimum description length

Here is the theorem that turns entropy from a summary statistic into a law of nature about descriptions. Shannon's **source coding theorem** says: you cannot compress a message below its entropy without losing information, and you can always get arbitrarily close to it. Entropy is not a rule of thumb for compression; it is the hard floor. No present or future algorithm, however clever, beats it on average for a given source.

*The shape of the argument.* You should see why this floor exists, because the reasoning leans directly on Chapter 11 and it is beautiful. Consider a source emitting a long block of  $n$  independent symbols. By the Law of Large Numbers from the previous chapter, the *empirical* frequencies of the symbols in almost every long block will be very close to their true probabilities; wildly atypical blocks are possible but vanishingly rare (this is concentration of measure again). Call the blocks whose frequencies match the true distribution the *typical* blocks. Two facts about them follow from a short calculation: there are only about  $2$  raised to the power  $nH$  of them, and each one has nearly the same probability, about  $2$  raised to the power minus  $nH$ . So almost all of the probability lives on this modest set of roughly  $2$ -to-the- $nH$  sequences. To name one member of a set of that size, you need about  $nH$  bits, which is  $H$  bits per symbol. You cannot do better, because you genuinely have

that many distinct typical outcomes to tell apart; and you need not do better, because the atypical sequences are so rare you can spend as many bits as you like on them without denting the average. (Strictly, this is the asymptotic equipartition property, and I am stating it informally; the honest version is an “arbitrarily close as  $n$  grows” statement, not an exact equality for finite blocks.) The floor is  $H$  because  $H$  is the log of the number of outcomes you cannot avoid distinguishing.

This is where your generative-equation intuition first bites. Suppose the data were produced by a simple rule, say  $y = 2x + \text{noise}$ . Then the data are *not* an arbitrary high-entropy source; almost all of their apparent content is predictable from the rule, and only the noise is genuinely uncertain. A compressor that *discovers* the rule can store the two numbers of the rule plus the small residual noise, instead of storing every data point. That is dramatic compression, and it works precisely because the rule captures the structure that dropped the entropy below the naive maximum. The search for a compact model *is* the search for a short description, and a short description that reproduces the data is *evidence that the system has structure*, because structure is exactly what a compressor can exploit and noise is exactly what it cannot. The deepest version of this idea, the length of the shortest *program* that outputs the data, and why that length is uncomputable, is the subject of Chapter 15. For now, keep the operational version: shorter faithful description means more structure found.

**Worked example: how far can you squeeze a lopsided stream?** You have one million values, each 0 or 1, with 90% zeros. Stored naively, that is one million bits. But the source is far from a fair coin, so its entropy is well below 1 bit per symbol. Compute it:  $H = -(0.9 \log_2 0.9 + 0.1 \log_2 0.1)$ . Now,  $\log_2 0.9 \approx -0.152$  and  $\log_2 0.1 \approx -3.322$ , so  $H \approx -(0.9)(-0.152) - (0.1)(-3.322) \approx 0.137 + 0.332 \approx 0.469$  bits per symbol. The source coding theorem says the whole file can be squeezed to about  $0.469 \times 1,000,000 \approx 469,000$  bits, roughly 59 kilobytes, and no further without loss. Against the naive one million bits, that is a compression ratio of about  $1 / 0.469 \approx 2.13$ . *What this adds to your map:* the compressibility of a dataset is not a property of your zip utility; it is a property of the source, fixed in advance by its entropy, and your best tool can only approach that number, never beat it. A sparse indicator vector (90% zeros) is compressible for the same reason a rare event is informative: its distribution is far from flat, and distance from flat is exactly what buys you bits.

## KL divergence: the cost of being wrong

Entropy is the cost of describing a source with a code perfectly matched to it. But you almost never have the perfect code, because you almost never know the true distribution; you have a *model* of it. What does it cost to describe reality with the wrong model? That cost has a name and a formula. The **Kullback-Leibler divergence**  $D_{\text{KL}}(P \parallel Q)$  measures the *extra* bits per observation you pay when you describe data that truly come from distribution  $P$  using a code optimized for a different distribution  $Q$ .

This is the exact information-theoretic content of model misspecification, and it turns “my model is a bit off” into a number. If your model  $Q$  equals the truth  $P$ , the divergence is zero: no wasted bits. The more  $Q$  misplaces its probability relative to  $P$ , the more bits you burn, because your code spends short codewords on outcomes the model thought were common and long codewords on outcomes that are actually common.  $D_{\text{KL}}$  is always at least zero, and it is zero only when  $Q$  matches  $P$  exactly; it is the penalty, in the hard currency of bits, for believing the wrong distribution.

Two facts connect this straight to your daily practice. First, **maximum likelihood estimation minimizes this divergence**: fitting by maximizing likelihood is, up to a constant that does not depend on your parameters, the same as choosing the model  $Q$  that is closest to the true  $P$  in KL.

When you fit a model, you are quietly minimizing wasted bits. Second, the **cross-entropy loss** you already train classifiers with decomposes cleanly:

$$H(P, Q) = H(P) + D_{\text{KL}}(P \parallel Q)$$

The cross-entropy is the entropy of the truth (a floor you cannot lower, fixed by the data) plus the KL divergence of your model from the truth (a penalty you *can* lower by fitting better). Minimizing cross-entropy loss is therefore identical to minimizing KL divergence, because the  $H(P)$  term does not move as you train. Every time you have watched a training loss curve descend, you were watching your model shed wasted bits, driving  $Q$  toward  $P$ .

**Worked example: the network with the lower loss.** You fit two models to the same data, a linear model and a neural network, and the network reports a lower cross-entropy loss. In information-theoretic terms this means the network's distribution  $Q$  is closer to the true  $P$  in KL divergence *on this data*: it wastes fewer bits per observation. Does that make it the better model? Not necessarily, and the framework tells you exactly what is missing. Cross-entropy on the *training* set can always be driven down by a model flexible enough to bend toward every point, including the noise, which is structure that does not belong to  $P$  at all. To decide, you need the loss on *held-out* data drawn from the same source, and you need the two models' complexities, their description lengths, because a lower training loss bought with a vastly longer model description is not compression; it is memorization wearing compression's clothes. That comparison, held-out fit weighed against description length, is Chapter 14's Occam's razor, and it is why this chapter is a prerequisite for that one.

This also gives the crisp information-theoretic reading of two things you do reflexively. **Overfitting** is the model encoding the training set's *noise* as if it were signal: it compresses the specific sample below the true source entropy by memorizing accidents, which is impossible to do for genuine structure, so the apparent gain evaporates on new data where the accidents differ. **Regularization** (L1, L2, dropout) is a penalty on the model's *description length*: it pushes toward shorter model descriptions, and a shorter model that still fits is, by the argument above, one that captured structure rather than noise. Regularization helps generalization because it is, underneath, a minimum-description-length principle: pay for the model's complexity in the same currency as its misfit, and prefer the total that is shortest.

**Worked example: regularization is a length penalty.** Make the bookkeeping explicit. Think of the total cost of a model as two bit-counts added together: the bits to *write down the model* (its parameters, to some precision) plus the bits to *write down the data given the model* (the residuals it fails to predict, which is the cross-entropy term). An unregularized fit minimizes only the second count and is happy to spend unlimited bits on the first: many large, finely tuned parameters, each of which costs bits to specify, in exchange for shaving the residuals. That trade is exactly overfitting seen through the meter, because the extra parameter bits are buying reductions in residual bits that are really noise, not structure. Now add an L1 or L2 penalty. It is a charge on the *first* count, the cost of describing the parameters: L2 makes large parameters expensive, L1 makes nonzero parameters expensive (which is why it drives some to exactly zero, deleting them from the description entirely). The optimizer now minimizes the *sum* of description length and misfit, which is the minimum-description-length objective in disguise. Generalization improves because the total-bits objective refuses to spend model-description bits unless they buy back *more* data-description bits than they cost, and only genuine structure, present in future data too, pays off at that exchange rate. Noise does not, so the penalty starves it. This is the same trade Chapter 14 will formalize as Occam's razor; you are meeting it here as a literal bit-count.

## Mutual information: what variables know about each other

Entropy and KL measure single distributions and the gap between them. The last quantity measures a *relationship*. The **mutual information**  $I(X; Y)$  measures how much knowing  $X$  tells you about  $Y$ , and it is symmetric: it is equally how much  $Y$  tells you about  $X$ . Written in terms of entropy,

$$I(X; Y) = H(X) - H(X|Y)$$

which reads: your uncertainty about  $X$ , minus your *remaining* uncertainty about  $X$  once you already know  $Y$ . The quantity  $H(X|Y)$ , the **conditional entropy**, is just the entropy of  $X$  computed after  $Y$  is revealed, averaged over the values  $Y$  can take; it is how uncertain  $X$  still is when  $Y$  is on the table. Subtract it from the original uncertainty  $H(X)$  and what is left is the uncertainty that  $Y$  removed, which is by definition what  $Y$  told you about  $X$ . If learning  $Y$  leaves your uncertainty about  $X$  untouched, then  $H(X|Y)$  equals  $H(X)$ , the difference is zero, the two are independent, and their mutual information is zero. If learning  $Y$  pins  $X$  down completely, then  $H(X|Y)$  is zero and the mutual information equals the full entropy of  $X$ . Everything in between is a partial constraint, measured in bits.

Mutual information is the honest, general measure of dependence, and it is strictly more powerful than correlation. Correlation measures only the *linear, monotone* part of a relationship; it asks whether large  $X$  tends to come with large  $Y$  along a straight line. Mutual information asks the far broader question of whether knowing one *constrains* the other *at all*, by any pattern, linear or not. This matters because biology is full of dependencies that correlation cannot see.

**Worked example: zero correlation, high dependence.** Let  $X$  take the values  $-2, -1, 1, 2$  with equal probability, and let  $Y = X \cdot X$  (so  $Y$  is 1 or 4). The correlation between  $X$  and  $Y$  is exactly zero: for every positive  $X$  there is an equally likely negative  $X$  with the same  $Y$ , so  $X$  and  $Y$  have no linear trend whatsoever. Yet the dependence is total in one direction: knowing  $X$  determines  $Y$  with certainty, and knowing  $Y$  cuts your uncertainty about  $X$  in half (it tells you the magnitude, leaving only the sign). The mutual information is high while the correlation is nil. The biological version is everywhere: enzyme activity against temperature is an inverted U, rising then falling, so activity and temperature can be strongly dependent yet nearly uncorrelated across a wide range; the same holds for anything with an optimum or a periodic driver. *What this adds to your map:* a correlation of zero is not evidence of independence; it is evidence of no *linear* dependence, and the two are different claims. When you need to know whether one variable carries information about another, correlation can lie by omission and mutual information cannot.

## The connection to modeling and compression

Now assemble the pieces into the chain that vindicates your original intuition. Each link is a quantity you now command.

1. A model is a *compressed description* of a system's behavior: it stands in for the data with fewer bits than the data themselves.
2. The quality of that compression is measured by how well the model predicts, which is measured by the KL divergence between the true distribution and the model. Better model, fewer wasted bits, tighter compression.
3. The best possible compression is bounded below by the system's *entropy*, its irreducible uncertainty. You cannot describe a source in fewer bits than its entropy, no matter how good the model.



4. If the system was actually generated by a *short rule*, its entropy is low and the compression can be extreme: the model can be small because most of the apparent content was predictable from the rule.
5. Finding the short rule is *better than* finding the best statistical compression, because the rule generalizes. It compresses not only the data you have seen but all future data the same process will emit.

This chain also reads a common experiment for you. Suppose a very simple model, a handful of parameters, fits your data almost as well as a lavishly complex one. What have you learned? By links 3 and 4, that the data-generating process has *low entropy relative to its apparent size*: most of what looked like content was predictable structure, so a short description already captures nearly all of it, and the complex model's extra capacity found almost nothing left to explain. The process is highly compressible, which is to say highly structured, which is to say close to being ruled by a short generative equation. That is a strong and encouraging diagnosis, and it is the opposite of the discouraging case where only a large model fits, signalling either a genuinely high-entropy source or a mechanism your model family cannot express. Your own frontier question, whether the *search space* of candidate rules could itself be made variable rather than fixed in advance, lives exactly here: it is the question of how to search efficiently for the short generative description when you do not know in advance which family it belongs to, and it is the doorway into Chapter 15, where the shortest program becomes the definition of complexity itself.

Point 5 is your insight, now stated precisely. Knowing the rule that produced the data is the ultimate compression because the rule has essentially zero KL divergence from the true process (up to the noise the rule does not pretend to predict), and it retains that zero divergence on data you have not yet collected. A *statistical* compression, a histogram, a kernel density estimate, a lookup table of observed frequencies, can approximate the distribution of the data you have, and can even hit the entropy floor on that sample, but it has memorized a shape rather than a mechanism. Show it a new regime and it fails, because it never held the generator. A *generative* compression holds the mechanism, so it extrapolates as far as the mechanism reaches. This is the difference between describing *what the data did* and describing *what the system does*, and it is the whole reason a two-parameter law can be worth more than a thousand-bin histogram that fits the sample better.

One consequence is worth stating as a warning, because it will sharpen your reading of any compressed result. A file compressed all the way to its entropy limit is, by construction, statistically indistinguishable from random noise: every remaining bit is genuinely unpredictable, or a compressor would have removed it. So you can learn *nothing* about the structure of the original data by staring at its optimally compressed form; all the structure has been spent, converted into the shortening itself. The structure did not vanish; it was *used*. This is the precise sense in which compression and understanding are the same coin: what you understand about a source, you can remove from its description, and what remains is exactly what you do not yet understand.

### Where you now stand

You came in with a currency-free intuition that understanding is compression. You leave with the currency. Information is surprise,  $-\log_2 p$  per event. Entropy,  $H(X) = -\sum p(x) \log_2 p(x)$ , is average surprise, and is simultaneously your uncertainty, your information gain, and the hard floor on description length. KL divergence is the bit-cost of a wrong model, the thing maximum likelihood minimizes and cross-entropy loss reports. Mutual information is dependence measured honestly, blind to no pattern. And the whole apparatus certifies your generative-equation instinct: the

shortest faithful description wins, and a rule beats a summary because a rule generalizes. The uncomputable limit of this idea waits in Chapter 15; its use in judging models waits in Chapter 14.

But there is a quiet assumption threaded through everything you just did, and it hands the next chapter its subject. Every entropy, every divergence, every mutual information was computed over *numbers or symbols that were simply handed to you*, as if the data arrive pre-numbered. They do not. Something had to attach numbers to the world before any of these bits could be counted, and that something makes choices, of scale and of units, that these formulas silently depend on. Change your units and a continuous entropy shifts; some information quantities survive a change of scale and others do not, exactly as some statistics survived Chapter 11's transformations and others were artifacts. So the debt this chapter incurs is the mirror image of the one it paid off: information theory tells you the cost of describing numbers, but not where the numbers came from, nor which operations on them mean anything. That is Chapter 13. Before you can trust a bit, you have to know how the world got its numbers, and measurement, it turns out, is the very first structure-preserving map in the entire pipeline.